

**TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN**



HỌ VÀ TÊN SINH VIÊN: Nguyễn Quang Hùng

MÃ SINH VIÊN: 22T1020142

TÊN ĐỀ TÀI

Tạo máy chủ web cục bộ với ESP32 để quản lý thiết bị

PHÁT TRIỂN ỨNG DỤNG IoT-2025-2026

GIẢNG VIÊN HƯỚNG DẪN: VÕ VIỆT DŨNG

HUẾ, THÁNG 4, NĂM 2026

MỤC LỤC

1. GIỚI THIỆU ĐỀ TÀI	1
1.1. Đặt vấn đề	1
1.2. Mục tiêu đề tài	2
1.3. Phạm vi nghiên cứu	2
2. CƠ SỞ LÝ THUYẾT	3
2.1. Tổng quan về ESP32	3
2.2. ESP32 Web Server	4
2.3. SPIFFS (SPI Flash File System)	5
2.4. Giao thức HTTP và WebSocket	6
2.5. Các cảm biến và thiết bị sử dụng	7
3. THIẾT KẾ HỆ THỐNG	8
3.1. Sơ đồ khối hệ thống	8
3.2. Sơ đồ kết nối phần cứng (Wokwi)	9
3.3. Thiết kế giao diện Web	10
3.4. Luồng hoạt động của hệ thống	11
4. MÔ PHỎNG TRÊN WOKWI	12
4.1. Sơ đồ mạch trên Wokwi	12
4.2. Danh sách linh kiện	13
4.3. Mô tả chức năng	14
5. TÍCH HỢP TELEGRAM BOT	15
5.1. Giới thiệu	15
5.2. Cách hoạt động	16
5.3. Các lệnh điều khiển	16
6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	17
6.1. Kết quả đạt được	17
6.2. Hạn chế	17
6.3. Hướng phát triển	17
7. TÀI LIỆU THAM KHẢO	17

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1. Đặt vấn đề

Trong thời đại Internet of Things (IoT) phát triển mạnh mẽ, việc quản lý và giám sát các thiết bị thông minh trong nhà, văn phòng hay nhà máy trở thành nhu cầu thiết yếu. Tuy nhiên, nhiều giải pháp IoT hiện tại phụ thuộc vào các dịch vụ đám mây (cloud), dẫn đến các vấn đề về:

- **Độ trễ (latency):** Dữ liệu phải đi qua Internet rồi mới quay về, gây chậm trễ trong điều khiển.
- **Bảo mật:** Dữ liệu truyền qua mạng công cộng có nguy cơ bị đánh cắp.
- **Phụ thuộc Internet:** Khi mất kết nối Internet, toàn bộ hệ thống ngừng hoạt động.
- **Chi phí:** Các dịch vụ cloud thường tính phí theo lưu lượng sử dụng.

Giải pháp: Xây dựng một **máy chủ web cục bộ (Local Web Server)** chạy trực tiếp trên vi điều khiển **ESP32**. Người dùng chỉ cần kết nối vào cùng mạng WiFi nội bộ, mở trình duyệt web và truy cập địa chỉ IP của ESP32 để quản lý toàn bộ thiết bị IoT mà **không cần Internet**.

1.2. Mục tiêu đề tài

- Xây dựng một Web Server chạy trên ESP32 trong mạng nội bộ (LAN/WiFi).
- Sử dụng **SPIFFS** để lưu trữ các file HTML, CSS, JavaScript trên bộ nhớ flash của ESP32.
- Tạo giao diện web cho phép:
- **Theo dõi (monitor):** Hiển thị dữ liệu cảm biến nhiệt độ, độ ẩm, ánh sáng theo thời gian thực.
- **Điều khiển (control):** Bật/tắt đèn LED, relay, quạt từ xa qua giao diện web.
- **Cấu hình (configure):** Thay đổi ngưỡng cảnh báo, tần suất đọc cảm biến.
- Gửi dữ liệu lên **ThingSpeak** để lưu trữ và phân tích dài hạn.

1.3. Phạm vi đề tài

- Sử dụng **Wokwi** để mô phỏng toàn bộ hệ thống (không yêu cầu phần cứng thực).
- Tập trung vào phân lý **thuyết, thiết kế hệ thống và sơ đồ mạch**.
- Các cảm biến không có trên Wokwi sẽ sử dụng **giá trị random** mô phỏng.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT (Trang 4-6)

2.1. Tổng quan về ESP32

ESP32 là vi điều khiển do **Espressif Systems** phát triển, được sử dụng rộng rãi trong các dự án IoT nhờ các đặc điểm nổi

Thông số	Chi tiết
CPU	Dual-core Xtensa LX6, xung nhịp 240 MHz

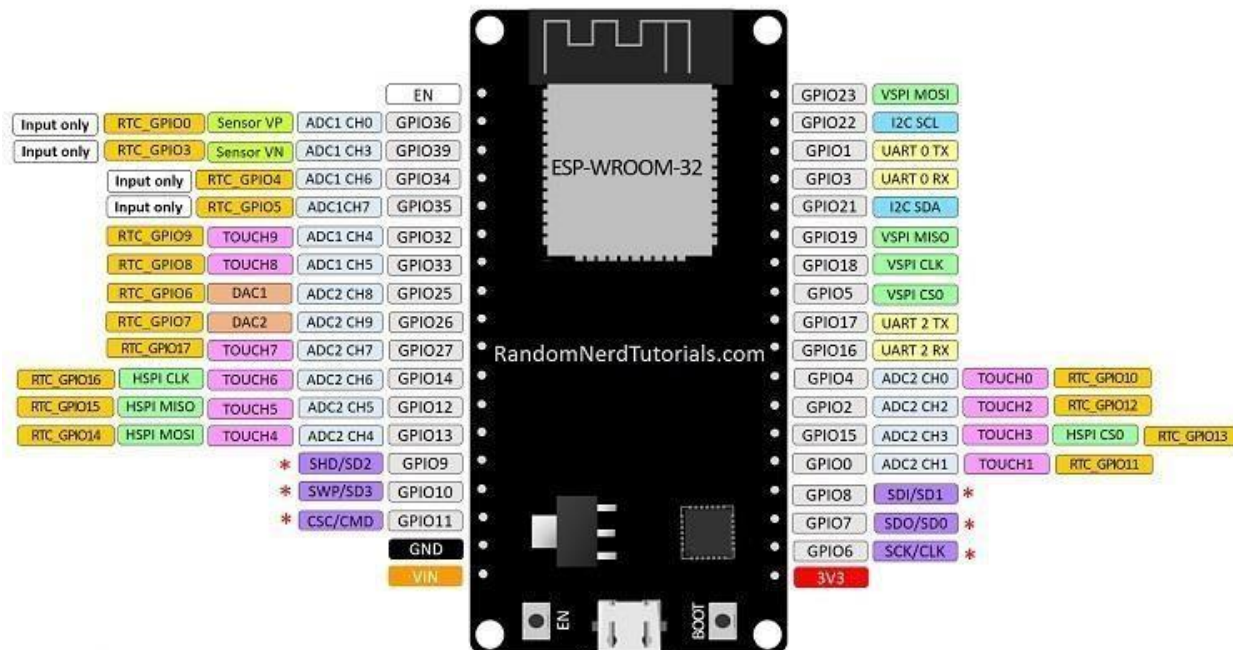
RAM	520 KB SRAM
Flash	4 MB (mặc định)
WiFi	802.11 b/g/n (2.4 GHz)
Bluetooth	BLE 4.2 và Classic
GPIO	34 chân GPIO đa chức năng
ADC	18 kênh ADC 12-bit
Giao tiếp	SPI, I2C, UART, I2S, PWM
Điện áp hoạt động	3.3V

Tại sao chọn ESP32 làm Web Server?

- Tích hợp sẵn WiFi → kết nối mạng không cần module ngoài.
- Đủ mạnh để chạy HTTP server, xử lý nhiều request đồng thời.
- Hỗ trợ SPIFFS → lưu file web trực tiếp trên flash.
- Chi phí thấp (~3-5 USD/module).
- Cộng đồng lớn, nhiều thư viện hỗ trợ.

ESP32 DEVKIT V1 – DOIT

version with 36 GPIOs

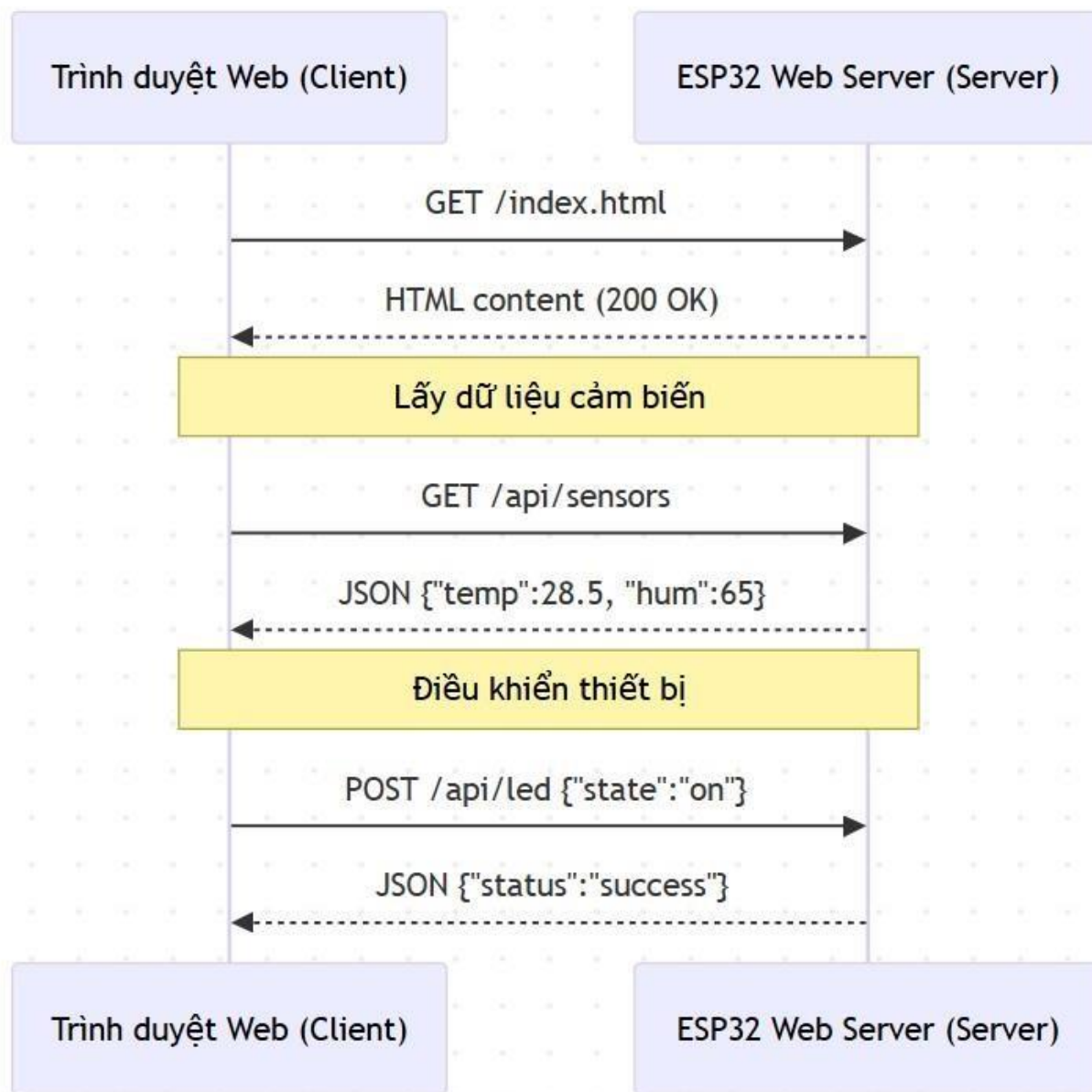


* Pins SCK/CLK, SDO/SD0, SDI/SD1, SHD/SD2, SWP/SD3 and CSC/CMD, namely, GPIO6 to GPIO11 are connected to the integrated SPI flash integrated on ESP-WROOM-32 and are not recommended for other uses.

2.2. ESP32 Web Server

Web Server là một phần mềm chạy trên thiết bị, lắng nghe các yêu cầu HTTP từ trình duyệt (client) và trả về nội dung web (HTML, CSS, JS, JSON...).

Cách hoạt động:



Các thư viện Web Server trên ESP32:

Thư viện	Đặc điểm
WebServer (built-in)	Đơn giản, đồng bộ, phù hợp project nhỏ
ESPAsyncWebServer	Bất đồng bộ, xử lý nhiều request đồng thời năng cao
AsyncWebSocket	Hỗ trợ WebSocket cho cập nhật real-time

Trong đề tài này, ta sử dụng **ESPAsyncWebServer** kết hợp **AsyncWebSocket** để đạt hiệu năng tốt nhất.

2.3. SPIFFS (SPI Flash File System)

SPIFFS là hệ thống file đơn giản được thiết kế cho các vi điều khiển có bộ nhớ flash hạn chế.

Đặc điểm:

- Lưu trữ file trên bộ nhớ flash nội bộ của ESP32 (thường 1.5 MB dành cho SPIFFS).
- Hỗ trợ đọc/ghi file với tên file tối đa 31 ký tự.
- **Không hỗ trợ thư mục con** (tất cả file nằm ở thư mục gốc /).
- Phù hợp để lưu: HTML, CSS, JavaScript, hình ảnh nhỏ, file cấu hình JSON.

Cấu trúc file trong SPIFFS:

/data

— index.html	(Giao diện chính)
— style.css	(Định dạng giao diện)
— script.js	(Xử lý logic phía client)
— config.json	(File cấu hình hệ thống)
— favicon.ico	(Icon trang web)

Lợi ích của SPIFFS so với nhúng HTML trong code:

Tiêu chí	Nhúng HTML trong code	Dùng SPIFFS
Dễ bảo trì	Khó sửa	Dễ sửa riêng từng file
Tách biệt code	Lẫn lộn C++ và HTML	Tách riêng rõ ràng
Kích thước	Giới hạn bởi RAM	Dùng flash (lớn hơn)
Cập nhật giao diện	Phải flash lại firmware	Chỉ cần upload file

bootloader
partition_table
nvs (24kB) 0x9000 - 0x10000
ota_data (8kB) 0x10000 - 0x12000
ota_0 (2Mb) 0x20000 0x220000
ota_1 (2Mb) 0x220000 0x420000
Storage (2Mb) 0x420000 0x620000
nvs_ext (64kB) 0x620000 0x630000

2.4. Giao thức HTTP và WebSocket

HTTP (HyperText Transfer Protocol)

- Giao thức **request-response**: Client gửi yêu cầu → Server phản hồi.
- **Stateless**: Mỗi request độc lập, server không nhớ trạng thái trước đó.
- Phù hợp cho: Tải trang web, gửi lệnh điều khiển.

WebSocket

- Giao thức **full-duplex**: Cả client và server đều có thể gửi dữ liệu bất kỳ lúc nào.
- **Persistent connection**: Kết nối được duy trì liên tục sau khi thiết lập.
- Phù hợp cho: Cập nhật dữ liệu cảm biến theo thời gian thực.

HTTP:

Client —Request—→ Server

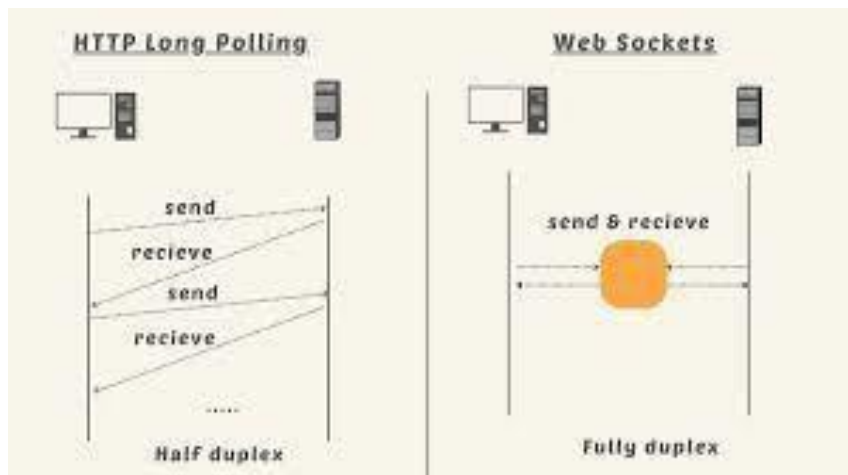
Client ←—Response— Server

(Kết nối đóng)

WebSocket:

Client ←=====→ Server

(Kết nối duy trì, truyền 2 chiều)

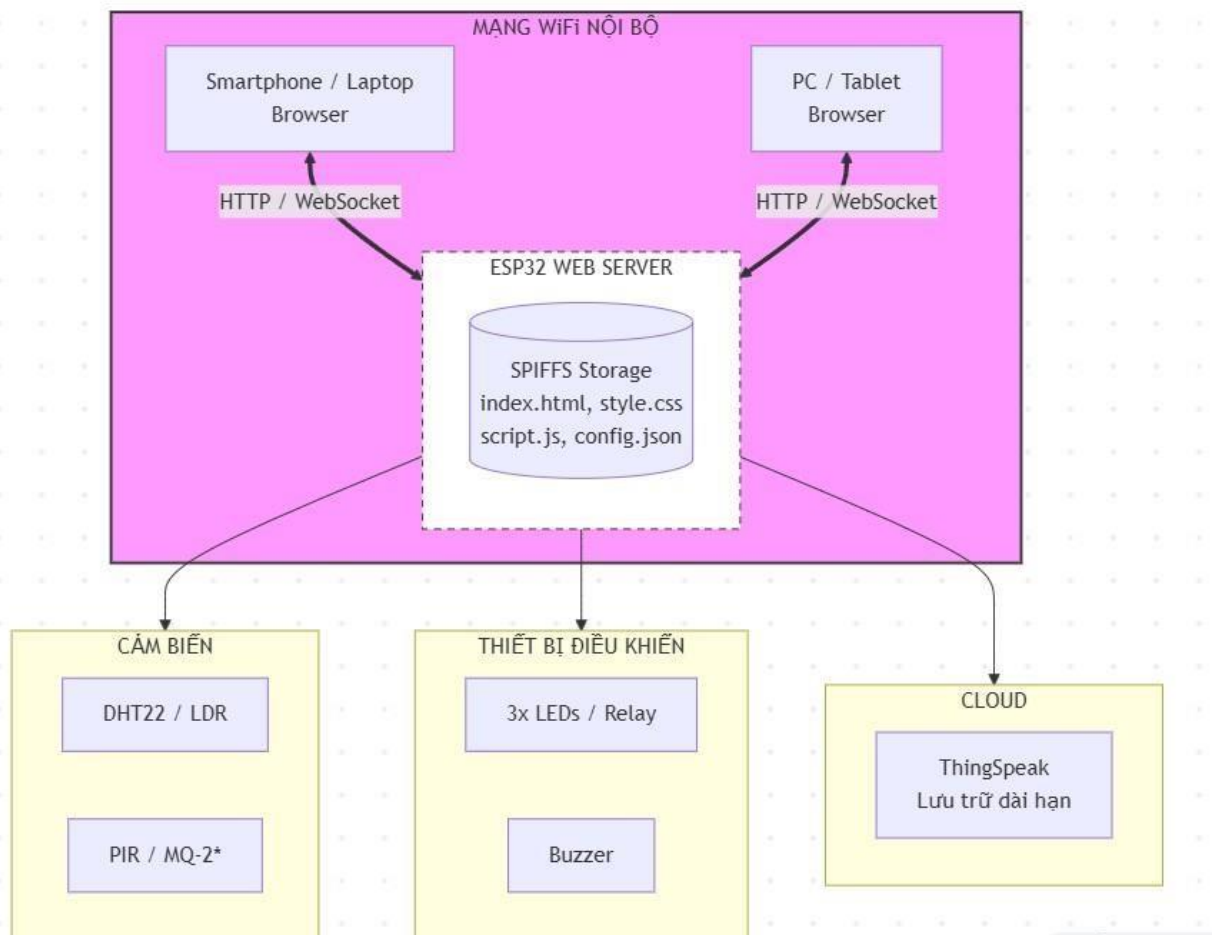


2.5. Các cảm biến và thiết bị sử dụng

STT	Thiết bị	Chức năng	Có trên Wokwi?
1	DHT22	Đo nhiệt độ và độ ẩm	Có
2	LDR (Quang trở)	Đo cường độ ánh sáng	Có
3	PIR Sensor	Phát hiện chuyển động	Có
4	LED (x3)	Đèn phòng khách, phòng ngủ, nhà bếp	Có
5	Relay Module	Điều khiển quạt/máy bơm	Có (dùng LED mô phỏng)
6	Cảm biến khí gas MQ-2	Phát hiện rò rỉ gas	Không → Random giá trị
7	Buzzer	Cảnh báo âm thanh	Có

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1. Sơ đồ khối hệ thống

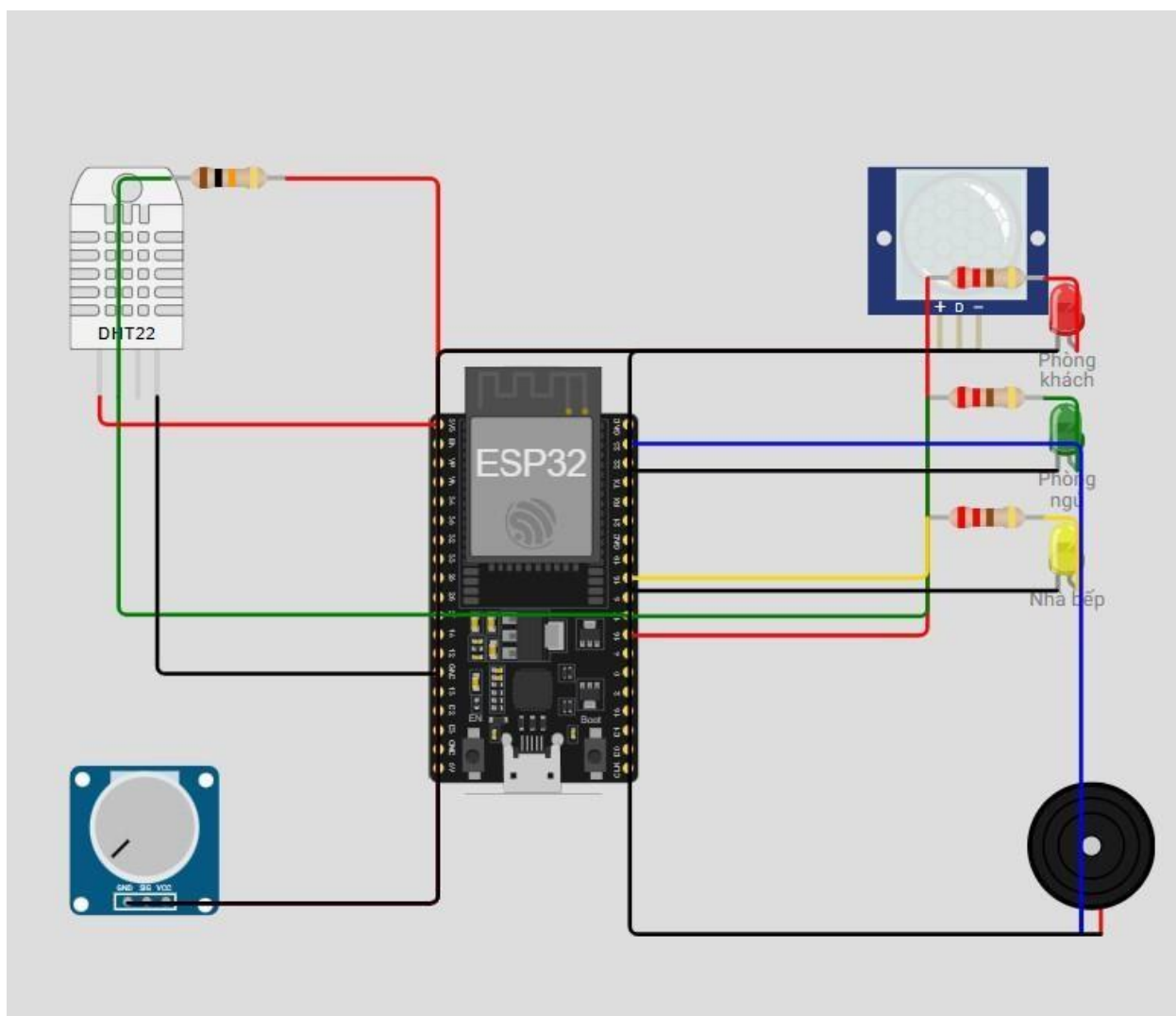


3.2. Sơ đồ kết nối phần cứng

Bảng kết nối chân:

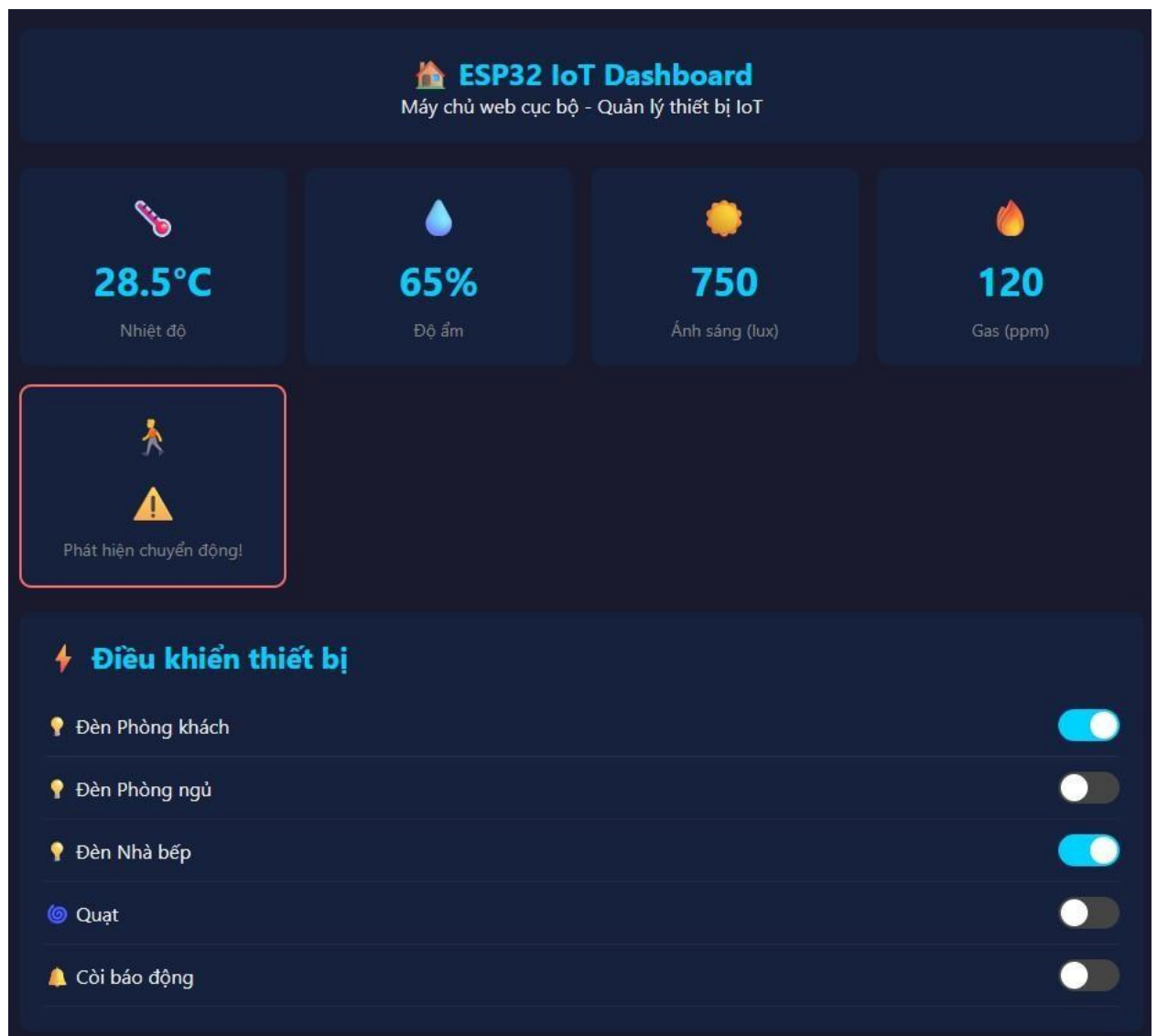
Thiết bị	Chân thiết bị	Chân ESP32	Ghi chú
DHT22	DATA	GPIO 4	Kèm điện trở kéo 10kΩ
DHT22	VCC	3.3V	
DHT22	GND	GND	
LDR (Potentiometer)	Signal	GPIO 34	Đọc ADC
PIR Sensor	OUT	GPIO 27	Input digital
LED Phòng khách	Anode	GPIO 16	Qua điện trở 220Ω
LED Phòng ngủ	Anode	GPIO 17	Qua điện trở 220Ω
LED Nhà bếp	Anode	GPIO 18	Qua điện trở 220Ω
Relay (Quạt)	IN	GPIO 19	Active LOW

Buzzer	Signal	GPIO 23	Cảnh báo gas/chuyển động
--------	--------	---------	--------------------------

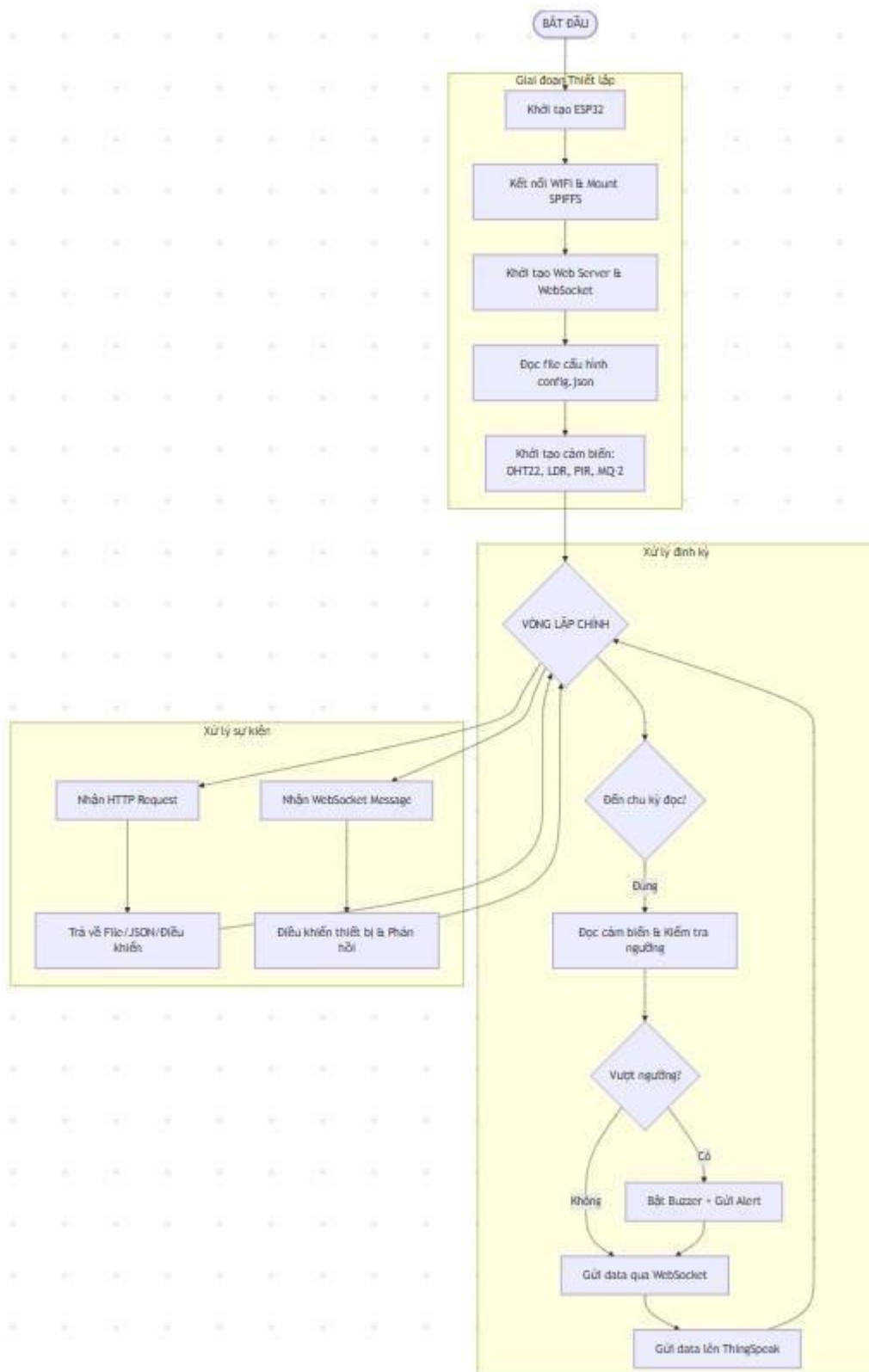


3.3. Thiết kế giao diện Web (trên SPIFFS)

Giao diện web được thiết kế responsive, hoạt động trên cả điện thoại và máy tính. **Layout giao diện:**

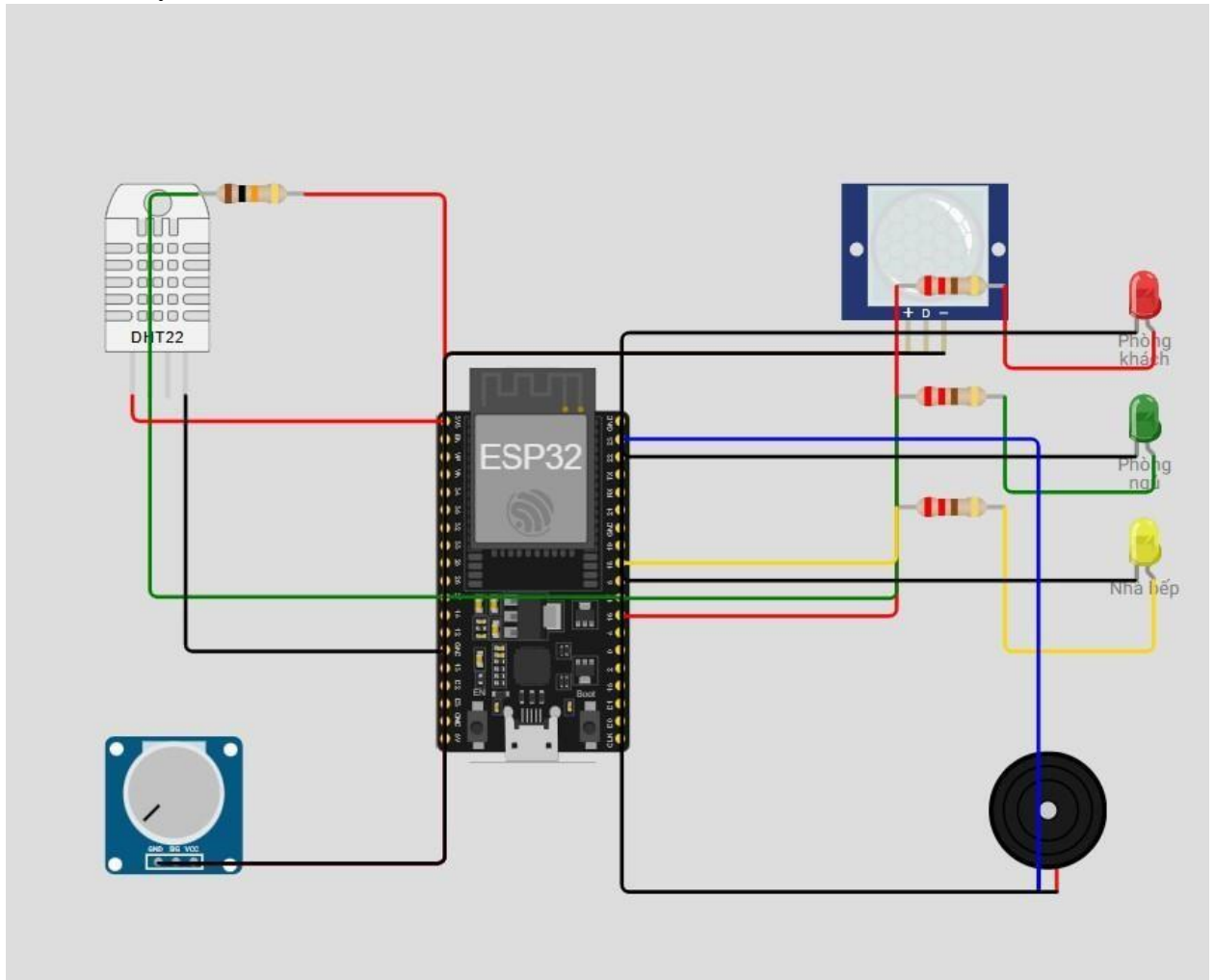


3.4. Luồng hoạt động của hệ thống



CHƯƠNG 4: MÔ PHỎNG TRÊN WOKWI

4.1. Sơ đồ mạch trên Wokwi



4.2. Danh sách linh kiện trên Wokwi

STT	Linh kiện	Số lượng	ID trong Wokwi	Vai trò
1	ESP32 DevKit C V4	1	esp	Vi điều khiển chính, Web Server
2	DHT22	1	dht1	Cảm biến nhiệt độ & độ ẩm
3	PIR Motion Sensor	1	pir1	Phát hiện chuyển động
4	Potentiometer	1	pot1	Mô phỏng cảm biến ánh sáng
5	LED Đỏ	1	led1	Đèn phòng khách

6	LED Xanh	1	led2	Đèn phòng ngủ
7	LED Vàng	1	led3	Đèn nhà bếp
8	Buzzer	1	bz1	Còi cảnh báo
9	Điện trở 220Ω	3	r1, r2, r3	Hạn dòng cho LED
10	Điện trở 10kΩ	1	r4	Kéo lên cho DHT22

4.3. Mô tả chức năng từng thành phần DHT22 - Cảm biến nhiệt độ và độ ẩm:

- Giao tiếp: Giao thức 1-Wire (Single Bus)
- Phạm vi đo nhiệt độ: -40°C đến 80°C ($\pm 0.5^\circ\text{C}$)
- Phạm vi đo độ ẩm: 0-100% RH ($\pm 2\%$)
- Tần suất đọc: tối thiểu 2 giây/lần
- Trong Wokwi: Click vào DHT22 → kéo thanh trượt để thay đổi nhiệt độ/độ ẩm **PIR**

Motion Sensor - Cảm biến chuyển động:

- Nguyên lý: Phát hiện bức xạ hồng ngoại từ cơ thể người
- Output: Digital HIGH khi phát hiện chuyển động
- Trong Wokwi: Click vào PIR → click nút để mô phỏng chuyển động **Potentiometer**

(mô phỏng LDR):

- Thay thế cho cảm biến ánh sáng (LDR) không có trên Wokwi
- Output: Giá trị analog 0-4095 (ADC 12-bit)
- Quy đổi: 0 = tối hoàn toàn, 4095 = sáng nhất **Cảm biến MQ-2 (Random giá trị):**
- Không có trên Wokwi → sử dụng hàm random(50, 500) để mô phỏng
- Giá trị < 300 ppm: An toàn (hiển thị xanh)
- Giá trị ≥ 300 ppm: Nguy hiểm (hiển thị đỏ, kích hoạt buzzer)

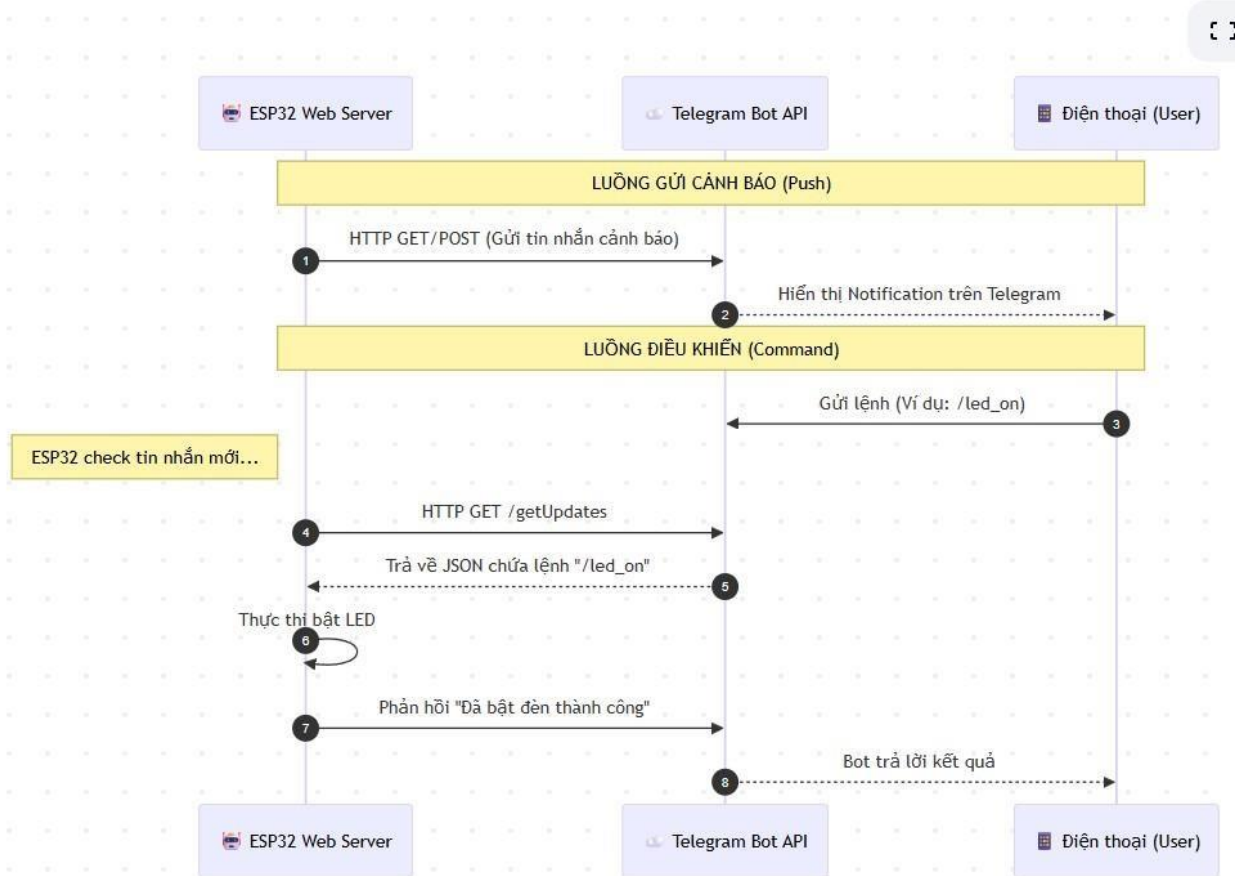
CHƯƠNG 5: TÍCH HỢP TELEGRAM BOT ĐỂ GỬI CẢNH BÁO 5.1. Giới thiệu Telegram Bot

Telegram Bot là chương trình tự động chạy trên nền tảng Telegram, cho phép:

- Gửi tin nhắn cảnh báo tự động đến người dùng
- Nhận lệnh điều khiển từ người dùng (bật/tắt thiết bị)
- **Miễn phí hoàn toàn**, không giới hạn số lượng tin nhắn
- API đơn giản, chỉ cần gửi HTTP GET request **Tại sao chọn Telegram thay vì ThingSpeak?**

- Không cần tạo tài khoản phức tạp
- Nhận thông báo **tức thì** trên điện thoại
- Có thể **điều khiển ngược** từ Telegram → ESP32
- Phù hợp với mục đích **cảnh báo khẩn cấp** (gas, chuyển động lạ)

5.2. Cách hoạt động



Luồng gửi cảnh báo:



Luồng nhận lệnh điều khiển:



5.3. Tạo Telegram Bot (Hướng dẫn)

Bước 1: Mở Telegram → tìm **@BotFather** → gửi lệnh /newbot

Bước 2: Đặt tên bot (ví dụ: ESP32_IoT_Manager_Bot)

Bước 3: BotFather trả về Bot

Token (dạng: 110201543:AAHdqTevCH1vGWJxfSeofSAs0K5PALDsaw)

Bước 4: Tìm **Chat ID** bằng cách chat với bot **@userinfobot** → lấy ID

5.4. Các lệnh Telegram Bot hỗ trợ

Lệnh	Chức năng	Phản hồi ví dụ
/status	Xem trạng thái tất cả cảm biến	🌡️ 28.5°C 💧 65% ☀️ 750 lux
/led1 on	Bật đèn phòng khách	Đèn phòng khách: BẬT
/led1 off	Tắt đèn phòng khách	Đèn phòng khách: TẮT

/led2 on	Bật đèn phòng ngủ	Đèn phòng ngủ: BẬT
/led3 on	Bật đèn nhà bếp	Đèn nhà bếp: BẬT
/fan on	Bật quạt	Quạt: BẬT
/alloff	Tắt tất cả thiết bị	Đã tắt tất cả thiết bị
/help	Xem danh sách lệnh	Danh sách lệnh...

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Đề tài đã trình bày thành công việc thiết kế hệ thống máy chủ web cục bộ trên ESP32 với các thành phần:

- ☒ **Lý thuyết nền tảng:** ESP32, Web Server, SPIFFS, HTTP, WebSocket
- ☒ **Thiết kế hệ thống:** Sơ đồ khối, bảng kết nối chân, luồng hoạt động
- ☒ **Mô phỏng Wokwi:** Sơ đồ mạch hoàn chỉnh với các cảm biến (DHT22, PIR, LDR) và thiết bị điều khiển (LED, Buzzer)
- ☒ **Giao diện Web:** Dashboard cho phép giám sát cảm biến và điều khiển thiết bị theo thời gian thực
- ☒ **Tích hợp Telegram Bot:** Gửi cảnh báo tự động và nhận lệnh điều khiển thiết bị từ xa qua Telegram

Ưu điểm của giải pháp:

- Hoạt động độc lập không cần Internet (cho phần điều khiển cục bộ qua Web Server)
- Phản hồi nhanh (< 100ms trong mạng LAN)
- Bảo mật cao (dữ liệu điều khiển không rời khỏi mạng nội bộ)
- Chi phí thấp (chỉ cần 1 ESP32 + cảm biến)
- Giao diện web responsive, tương thích đa thiết bị
- Telegram Bot cho phép nhận cảnh báo và điều khiển từ bất kỳ đâu có Internet

6.2. Hạn chế

- SPIFFS có dung lượng hạn chế (~1.5 MB) → không lưu được file lớn
- ESP32 Web Server hỗ trợ tối đa ~5-10 client đồng thời
- Cảm biến MQ-2 phải dùng giá trị random do Wokwi không hỗ trợ

- Chưa có cơ chế xác thực (authentication) trên giao diện web
- Telegram Bot yêu cầu kết nối Internet → không hoạt động khi mất mạng ngoài 6.3.

Hướng phát triển

Hướng phát triển	Mô tả
Thêm xác thực	Thêm login page với username/password cho Web Server
OTA Update	Cập nhật firmware từ xa qua giao diện web
Dùng LittleFS	Thay SPIFFS bằng LittleFS (hiệu năng tốt hơn)
MQTT	Tích hợp MQTT broker nội bộ trên ESP32
Nhiều ESP32	Mở rộng kiến trúc master-slave với nhiều ESP32
ThingSpeak/Blynk	Thêm lưu trữ dữ liệu dài hạn trên cloud để phân tích
mDNS	Dùng mDNS để truy cập bằng tên (esp32.local) thay vì IP
Telegram nâng cao	Thêm gửi ảnh, inline keyboard, lịch sử cảnh báo

TÀI LIỆU THAM KHẢO text

- [1] Espressif Systems, "ESP32 Technical Reference Manual", https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [2] Random Nerd Tutorials, "ESP32 Web Server using SPIFFS", <https://randomnerdtutorials.com/esp32-web-server-spiiffs-spi-flash-file-system/>
- [3] Random Nerd Tutorials, "ESP32 WebSocket Server", <https://randomnerdtutorials.com/esp32-websocket-server-arduino/>
- [4] Random Nerd Tutorials, "ESP32 Telegram: Send Messages & Control Outputs", <https://randomnerdtutorials.com/telegram-control-esp32-esp8266-nodemcu-outputs/>
- [5] Telegram Bot API Documentation, <https://core.telegram.org/bots/api>
- [6] Wokwi Documentation, "ESP32 Simulation", <https://docs.wokwi.com/guides/esp32>

- [7] Kolban, N. (2018). "Kolban's Book on ESP32", <https://leanpub.com/kolban-ESP32>
- [8] Espressif, "SPIFFS Filesystem", <https://docs.espressif.com/projects/espidf/en/latest/esp32/api-reference/storage/spiffs.html>